

SCHEME AND EXCEPTIONS

COMPUTER SCIENCE MENTORS 61A

November 11 – November 14, 2025

1 Scheme

1. What will Scheme output? Draw box-and-pointer diagrams to help determine this. (Ask your mentor if you're unsure what's going on. You aren't expected to understand this completely on your own.)

```
scm> (cons 1 (cons 2 nil))
```

```
scm> (cons 1 '(2 3 4 5))
```

```
scm> (cons 1 '(2 (cons 3 nil)))
```

```
scm> (cons 1 (2 (cons 3 nil)))
```

```
scm> (cons 3 (cons (cons 4 nil) nil))
```

```
scm> (define a '(1 2 3))
```

```
scm> a
```

```
scm> (car a)
```

```
scm> (cdr a)
```

```
scm> (car (cdr a))
```

How can we get the 3 out of a?

2. What will Scheme output?

```
scm> (define c 4)
```

```
scm> ((define (x) 1))
```

```
scm> (x)
```

```
scm> ((lambda (x y) (+ c)) 1 2)
```

```
scm> (eval 'c)
```

```
scm> '(cons 1 nil)
```

```
scm> (eval (list 'if '(even? c) 1 2))
```

```
scm> (let ((a (+ 3 1)) (b 3)) (+ a b) (/ a b))
```

3. Define `six-sevens`. Return the number of times that 7 follows 6 in the list.

```
> (six-sevens '(4 6 7 6 0 7))
1
> (sixty-ones '(7 6 7 4 6 7 6 0 7))
2
> (sixty-ones '(6 7 6 7 4 6 7 6 0 7))
3

(define (six-sevens lst)
  (cond (_____ )
        (_____ )
        (else _____)))
```

4. Implement `waldo`. `waldo` returns `#t` if the symbol `waldo` is in a list.

```
scm> (waldo '(1 4 waldo))
#t
scm> (waldo '())
#f
scm> (waldo '(1 4 9))
#f

(define (waldo lst))
```

5. **Extra challenge:** Define `waldo` so that it returns the index of the list where the symbol `waldo` was found (if `waldo` is not in the list, return `#f`).

```
scm> (waldo '(1 4 waldo))
2
scm> (waldo '())
#f
scm> (waldo '(1 4 9))
#f

(define (waldo lst)
```

```
)
```

2 Exceptions

1. `list_sum` is a function that adds up the elements of a list and returns the sum. However, we have mixed types in our list! Write `list_sum` such that rather than using any `if` or `for` statements, it uses exceptions to catch these errors.

```
def list_sum(lst):
    '''
    >>> list_sum([1, 2, 3, 4, 5])
    15
    >>> list_sum([1, '2', 3, '4', 5])
    9
    >>> list_sum(['1', '2', 3, 4, 5])
    12
    '''
    i = 0
    total = 0
    while True:
```

```
try:
    _____
except _____:
    return _____
except _____:
    _____
    continue
i += 1
```